

```
sqlx::migrate!().run(<&your_pool OR &mut your_connection>).
await?;
```

Note that the compiler won't pick up new migrations if no Rust source files have changed. You can create a Cargo build script to work around this with ``sqlx migrate build-script`` (see <https://docs.rs/sqlx/0.5/sqlx/macro.migrate.html>).

Now you have to go to the above file and add your SQL statements. Once you have done that, you can set up the database:

```
▫ sqlx database setup
Applied 20211223133946/migrate initial setup (27.149609ms)
```

Next, let's add the required dependencies to the *Cargo.toml* file:

```
[dependencies]
anyhow = "1.0.51"
sqlx = { version = "0.5.9", features = [ "runtime-tokio-
rustls", "postgres" ] }
tokio = { version = "1", features = ["full"] }
```

And you can modify the *main.rs* code to do the following things:

- Set up a connection pool
- Pass credentials to get a connection from the pool

```
use sqlx::{PgPool};
use anyhow::*;
use sqlx::postgres::PgPoolOptions;

#[derive(Clone)]
pub struct DBApplication {
    pool: PgPool
}

impl DBApplication {
    pub async fn new(config: String) -> Result<DBApplication> {
        let pool = PgPoolOptions::new()
            .max_connections(5)
            .connect(&config)
            .await?;
        Ok(DBApplication { pool })
    }
}

#[tokio::main]
async fn main() -> Result<> {
    let db = DBApplication::new("postgres://postgres:postgres@
localhost/sqlx-demo".into()).await?;
```

```
println!("Connection acquired!");
Ok(())
}
```

You can confirm if this works by running the command *cargo run*:

```
▫ cargo run
Finished dev [unoptimized + debuginfo] target(s) in 0.04s
Running `target/debug/sqlx-demo`
Connection acquired!
```

Domain types

We have two tables and, usually, we would want to model them into Rust structs. The easy way of doing this is to use *derive* macros:

```
#[derive(sqlx::FromRow, Debug)]
pub struct Person {
    id: i64,
    name: String,
    age: i16,
    email: String
}
```

The usual convention is to have two sets of records -- one for inserting and the other for fetching. I use the *New* prefix to distinguish them:

```
pub struct NewPerson {
    pub name: String,
    pub age: i16,
    pub email: String
}
```

Another nice option is to use the *query_as!* macro, as it doesn't need the above *derive* macro. I will be using that later in the demo.

Transactions

Let's write a function to create a person, and let's use a transaction to do the same:

```
pub async fn create_person(&self, person: NewPerson) ->
Result<Person> {
    let mut transaction = self.pool.begin().await?;
    let person = sqlx::query_as("INSERT INTO person(name,
age, email) VALUES ($1, $2, $3) RETURNING *;")
        .bind(person.name)
        .bind(person.age)
        .bind(person.email)
```